All being well, you should see the following output:

```
$ curl http://localhost:8080/index.php
<!DOCTYPE html>
<html>
  <head>
  <title>Twig include() demo.</title>
</head>
...
```

The call to Twig's `include()` show in the example demonstrates the argument order. Remember, the file path is always required as the first argument, followed by an optional mapping – the keys of this map become context variables in the sub-template. Finally, the three named parameters follow in any order, all of which are optional. In our case, we specified `with_context: false`, which means the only context data `head.html.twig` has to work with is the explicitly supplied `title` variable.

Twig's template inclusion mechanism is a great way to specialize parts of your page, allowing you to define different versions of the same content that can be easily switched in and out with a little logic and some flags set in the data passed to the `::render()` call. It doesn't particularly lend itself to repeatable units of content… well actually you *could* abuse `include()` to, say, render an array of web cards or suchlike; `include()` will get the job done. But there is a better way…

Twig Macros

It is possible to define reusable units of HTML in regular templates and insert them as many times as needed using a function-like syntax. Twig calls this feature *macros*, and they work like this:

```
// index.php
print $twig->render('macro-demo.html.twig');
```

```
{# views/macro-demo1.html.twig #}
{% macro listItem(text) %}
  <li>{{ text }}</li>
{% endmacro %}
```